

mCOLD Device Protocol

เวอร์ชัน 0.2 (ฉบับร่าง) · 2026-10-07 · firmware 0.7.0-dev.40

สถานะ: เสนอ (protocol 1) ปรับปรุง 7 ต.ค. 2026 เขียนโดยฝั่ง firmware ให้ทีมแอปปรับหรือขอแก้ไข ยังไม่ถือว่าสุดท้ายจนกว่าทีมแอปจะรับ โดยเฉพาะ UUID ที่จะตรงเมื่อแอปทำตามแล้ว

JSON คำขอและคำตอบชุดเดียวกันใช้ได้กับทุกช่องทาง ตอนนี้ใช้ทาง BLE และ USB serial (คำสั่ง rpc ใน console สำหรับทดสอบ) ภายหลังจะเพิ่มช่องทางอื่นโดยไม่ต้องเปลี่ยนรูปแบบ

สิ่งที่เปลี่ยนในเวอร์ชัน 0.2

เรื่อง	เดิม	ตอนนี้
ชื่อ trip	trip (เลขนับในกล่อง เช่น 14)	trip_id (UUID ที่กล่องสร้างเอง) + trip_date (YYYYMMDD) + trip_number (เลขรันของวัน)
ACK จาก server	{"trip":T,"upto":S}	{"trip_id":"<uuid>","upto":S}
คำสั่ง BLE ที่อ้าง trip	"trip": 14	"trip_id": "<uuid>" (MARK_DELIVERED, GET_TRIP_SUMMARY, READ_LOG_CHUNK)
คำตอบที่บอก trip	"trip": 14	trip_id, trip_date, trip_number
คอลัมน์ในแถวและ CSV	trip, seq, sn, ...	trip_id, trip_date, trip_number, seq, ... (MQTT ไม่มี sn, utc, detail; timestamp เป็น Unix)
รายการ Wi-Fi	ส่งเฉพาะที่มี เรียงตามชื่อ	ส่งครบ 5 ช่อง (slot 1-5, ช่องว่าง ssid: null) แกะหรือลบช่องไหนก็ได้ด้วยเลขช่อง
trip เก่า (ก่อน dev.34)	ส่งขึ้น server ได้	ไม่ส่งและไม่ปรากฏใน LIST_TRIPS เพราะไม่มี trip_id

เลขนับในกล่องยังมีอยู่ข้างใน แต่ไม่ออกมาข้างนอกอีก

1. หากกล่อง

1. แอปอ่านแท็ก NFC เป็น NDEF record ชนิด MIME application/json :

```
{"sn": "mCDV1-L0169-1069-002", "ble": "28:84:85:27:9B:86", "key": "571ab30e58225355ccfc9b010a935f65"}
```

sn และ ble มีตั้งแต่ช่วง bring-up ส่วน key คือรหัสยืนยันตัวปัจจุบันของกล่อง (ดูหัวข้อ Authorization ในข้อ 2) กล่องปรับข้อมูลในแท็กเอง โดยเขียนเฉพาะไบต์ที่เปลี่ยน ทีมแอปเป็นเจ้าของรูปแบบนี้ firmware จะเพิ่ม URI record (สำหรับแบนเนอร์บน iOS) เมื่อมี URL ให้ใส่

- 1. การแตะแท็กปลุก BLE ของกล่องด้วย: กล่องประกาศตัว (advertise) 60 วินาที
- 2. **iOS ไม่เห็น BLE MAC** แอปหากกล่องจากชื่อที่ประกาศ ซึ่งคือ SN และกรองด้วย service UUID ส่วน MAC ใน NDEF ใช้ตัดสินเมื่อมีหลายตัวเท่านั้น ตามที่ทีมแอปยืนยัน

การประกาศ: flags + service UUID แบบ 128 บิตใน advertisement, ชื่อเต็ม (SN) ใน scan response, ช่วง 20-40 ms ระหว่างที่หน้าต่างเปิด

2. BLE GATT

มี primary service เดียว ทุก UUID ใช้ฐาน d2a5xxxx-3818-4667-a205-b3ed9c37b8a5

UUID	ชื่อ	คุณสมบัติ	ความปลอดภัย
d2a50000-...	mCOLD service		
d2a50001-...	INFO	read	เปิด
d2a50002-...	STATUS	read, notify	เปิด
d2a50003-...	COMMAND	write	เปิด แต่สิ่งที่สั่งได้ขึ้นกับ AUTH
d2a50004-...	RESPONSE	notify	เปิด
d2a50005-...	EVENT	notify	เปิด

- **INFO** คือผลของ GET_INFO (ข้อ 4) แอปเช็คเวอร์ชัน protocol ได้ก่อนส่งอะไร
- **STATUS** คือผลของ GET_STATUS แจ้ง (notify) เมื่อสถานะ trip หรือ alarm เปลี่ยน และหลังบันทึกแต่ละครั้ง
- **COMMAND** รับคำขอ **RESPONSE** ส่งคำตอบกลับ
- **EVENT** แจ้งเหตุการณ์ของ trip และ alarm: {"ev": "ALARM_RAISE", "alarm": "TEMP_HIGH"}, ALARM_CLEAR, ALARM_ACK, TRIP_START, TRIP_STOP
- **SETTINGS** มากับ EVENT เอง (ตกลง 5 ต.ค.): กันที่แอป subscribe, อีกครั้งเมื่อ session ผ่าน AUTH และเมื่อมีการตั้งค่าเปลี่ยน (จากแอป server หรือ console) ทำให้หน้าตั้งค่าของแอปตรงกับกล่องเสมอ: {"ev": "SETTINGS", "config": {...ทุกค่า...}, "editable": [...], "trip_locked": [...], "server_rev": 1, "network": {...}} ส่วน network (Wi-Fi แต่ละเครือข่ายพร้อม DHCP หรือ IP คงที่ และ OTA base ไม่มีรหัสผ่าน) จะมีหลัง AUTH เท่านั้น ขนาดราว 1.1 KB จึงมาเป็นหลาย fragment

Authorization: แตะเพื่อยืนยันตัว

ตกลง 2 ต.ค. แทนการพิมพ์รหัสบนจอ **ไม่ต้อง pair BLE** มือถือไม่ขึ้นหน้าต่างของระบบ และไม่ต้องพิมพ์อะไร

ใครอยู่ในระยะ BLE ก็อ่าน INFO และ STATUS ได้ ส่วนคำสั่งที่เปลี่ยนค่า (มีเครื่องหมาย ✎ ในข้อ 4) ต้องมี session ที่ยืนยันตัวแล้ว ซึ่งยืนยันได้ด้วยการพิสูจน์ว่าแอปอ่านแท็ก NFC มา ซึ่งอ่านได้ในระยะไม่กี่เซนติเมตร จึงต้องถือกล่องอยู่

1. แตะ: อ่าน key (16 ไบต์ เป็น hex) จาก record ของแท็ก
2. เชื่อม BLE และอ่าน INFO ในนั้นมี nonce เลขสุ่ม 16 ไบต์ (hex) ใหม่ทุกการเชื่อมต่อ
3. ส่ง {"id": "...", "cmd": "AUTH", "proof": "<hex>"} โดย proof = HMAC-SHA256(key, nonce) คำนวณจาก **ไบต์ดิบ**ของทั้งสองค่า (ไม่ใช่ข้อความ hex) และส่งผลเป็น hex 64 ตัว
4. {"ok": true} คือการเชื่อมต่อนี้สั่งงานได้จนกว่าจะปิด ถ้าได้ NOT_AUTHORIZED ให้แตะใหม่แล้วลองอีก

ข้อควรระวัง (จากการทดสอบแอปวันที่ 7 ต.ค.): ช่องที่ AUTH อ่านคือ proof ไม่ใช่ key ถ้าส่ง {"cmd": "AUTH", "key": "..."} กล่องจะตอบ NOT_AUTHORIZED เสมอ และห้ามส่ง key ทาง BLE ไม่ว่ากรณีใด

key ไม่เคยเดินทางผ่าน BLE คนดักรับสัญญาณเห็นแค่ nonce กับ proof ซึ่งใช้กับการเชื่อมต่ออื่นไม่ได้

key เปลี่ยนเมื่อกล่องรีเซ็ต และ 2 นาทีหลังจบ session ที่ใช้ key นั้น แตะหนึ่งครั้งได้หนึ่ง session (ถ้าสิงห์หลุดกลางทาง เชื่อมกลับภายใน 2 นาทีด้วยการแตะเดิมได้) กล่องเขียน key ใหม่ลงแท็กก่อนเริ่มรับ แท็กจึงไม่เคยมี key ที่กล่องจะปฏิเสธ การแตะ NFC เฉยๆ ไม่ทำให้ key เปลี่ยน

ตัวอย่างจาก test client ของ firmware:

```
key 7b3066da5096846781a0b5c1d0f6699b
nonce 951c0f73... (จาก INFO)
proof = HMAC-SHA256(key, nonce) -> {"cmd": "AUTH", "proof": "..."} -> ok
```

Fragments (การแบ่งข้อความ)

ข้อความยาวกว่าการเขียนหรือการ notify ครั้งเดียว ทั้งสองทิศจึงใช้กรอบเดียวกันโดยไม่ขึ้นกับ MTU:

```
byte 0 flags: bit 7 FIRST, bit 6 LAST, bits 0-5 ลำดับ fragment (mod 64)
byte 1.. ส่วนหนึ่งของ JSON แบบ UTF-8
```

ข้อความหนึ่งคือ fragment FIRST ถึง LAST ตามลำดับ (BLE write with response และ notify เรียงลำดับอยู่แล้ว) ข้อความที่มี fragment เดียว ตั้งทั้งสองบิต fragment ที่ลำดับผิด หรือรวมเกิน 4,096 ไบต์ จะทิ้งทั้งข้อความ และกล่องตอบ BAD_REQUEST ด้วย id 0 คำตอบถูกตัดให้พอดี MTU ที่ตกลงกัน (MTU - 3 ไบต์ของ ATT - 1 ไบต์ของ flags)

3. รูปแบบข้อความ

คำขอ:

```
{"id": 17, "cmd": "START_TRIP", "low": 2.0, "high": 8.0}
```

คำตอบ:

```
{"id": 17, "ok": true, "trip_id": "95518e06-1b3b-494f-b4c5-284ae94f5424", "trip_date": "20261007", "trip_number": 2}
{"id": 17, "ok": false, "err": "ALREADY_ACTIVE", "msg": "a trip is already running"}
```

- id: แอปเลือก ค่า 1 ถึง 4294967295 ไม่ซ้ำกันต่อคำขอ คำตอบส่ง id กลับมา
- **Idempotency:** ทุกคำสั่งที่เปลี่ยนค่า (มี ✎) จำ id ของตัวเอง ส่ง id เดิมอีกครั้งจะได้คำตอบเดิมโดยไม่ทำซ้ำ ดังนั้นเมื่อหมดเวลาหรือสิงห์หลุด แอปลองใหม่ด้วย id เดิม ไม่ใช่ id ใหม่ กล่องจำ 8 รายการล่าสุดใน RAM ส่วน START_TRIP จำใน flash ด้วย ลองซ้ำหลังกล่องรีเซ็ตก็ได้ trip เดิม **อย่าใช้ id ซ้ำกับคำสั่งคนละอัน** เพราะจะได้คำตอบของคำสั่งเก่า

- ช่องที่กล่องไม่รู้จักในคำขอจะถูกข้าม เพิ่มช่องได้
- ส่ง proto ได้ ถ้าเวอร์ชันที่กล่องไม่รองรับจะตอบ UNSUPPORTED_PROTO

รหัส error:

err	ความหมาย
BAD_REQUEST	ไม่ใช่ JSON, ไม่มี id หรือไม่มี cmd
UNKNOWN_CMD	กล่องไม่รู้จักชื่อ cmd (ต้องเป็นตัวพิมพ์ใหญ่ตามข้อ 4 เช่น APPLY_CONFIG ไม่ใช่ set)
BAD_ARGS	ขาดหรือเกินช่วงของ argument
NOT_AUTHORIZED	ต้องผ่าน AUTH ก่อน หรือ AUTH ไม่ผ่าน
UNSUPPORTED_PROTO	proto ใหม่เกินไป
ALREADY_ACTIVE / NOT_ACTIVE	สถานะ trip ไม่ให้ทำ
LOG_FULL , NO_LOG , FLASH	พื้นที่เก็บทำไม่ได้
NOT_SUPPORTED	กำหนดไว้แล้ว แต่ firmware นี้ยังไม่มี

หน่วย: อุณหภูมิเป็น °C ตัวเลข, เวลาเป็นวินาที Unix UTC พร้อมคุณภาพแยกต่างหาก (none , rtc , gnss , host , ntp), ระยะเวลาคือวินาที ค่าที่กล่องไม่มีจะไม่ใช่หรือเป็น null ไม่เคยเป็น 0

ตัวตนของ trip (ตกลง 7 ต.ค.)

ช่อง	ตัวอย่าง	ความหมาย
trip_id	95518e06-1b3b-494f-b4c5-284ae94f5424	UUID (v4) ที่กล่องสร้างตอนเริ่ม trip ทำให้ server ผูก trip กับ sn ได้โดยไม่ซ้ำกัน
trip_date	"20261007"	วันที่ท้องถิ่นตอนเริ่ม trip รูปแบบ YYYYMMDD เป็นข้อความ (ถ้ากล่องไม่รู้เวลาตอนเริ่มจะเป็น "00000000")
trip_number	2	เลขรันของวันนั้น เริ่มที่ 1 และเริ่มใหม่เมื่อวันเปลี่ยน

ใช้ trip_id เป็นตัวอ้างอิงในทุกคำสั่งและ ACK ส่วน trip_date กับ trip_number ไว้ให้คนอ่าน เช่น "trip ที่ 2 ของวันที่ 7 ต.ค."

4. คำสั่ง

คำสั่ง	Argument	ผลลัพธ์
GET_INFO		proto , sn , fw , hw , idf , boot , uptime_s ; ทาง BLE มี nonce และ authorized ด้วย
AUTH	proof (hex)	ยืนยันตัวการเชื่อมต่อนี้ (ข้อ 2)
GET_STATUS		ดูด้านล่าง
GET_CONFIG		config : ทุกค่าพร้อมค่าปัจจุบัน
SET_CONFIG	key , value	ตั้งค่าหนึ่งตัว ทาง BLE ตั้งได้เฉพาะ key ใน docs/device-settings.md ค่าที่เลือกระหว่าง trip ตั้งไม่ได้ขณะ trip วิ่ง (key ในที่นี้คือชื่อค่าที่ตั้ง ไม่ใช่รหัสจาก NFC)
SET_TIME	utc (Unix วินาที)	quality
START_TRIP	low , high (°C); hyst (°C, 0.5), dwell_s (300)	trip_id , trip_date , trip_number
STOP_TRIP		trip_id , trip_date , trip_number
ACK_ALARM		alarms ที่ยังค้าง

คำสั่ง	✎ Argument	ผลลัพธ์
LIST_TRIPS		trips: [{"trip_id", "trip_date", "trip_number", "last_seq", "sent"}] เรียง เก่าก่อน; sent คือ server ได้ทุกแถวแล้ว; trip ที่ไม่มี trip_id ไม่อยู่ในรายการ
APPLY_CONFIG	✎ config, wifi (add: แต่ละรายการ มี slot ได้, del: เลขช่อง หรือชื่อ), ota_base (ใส่ ตัวไหนก็ได้; mqtt ปฏิกิริยา: ตั้งทาง console เท่านั้น)	applied, errors: เอกสารการตั้งค่าตาม docs/device-settings.md ชุดเดียวกับที่ server ส่งที่ mcold/v1/<sn>/config เฉพาะ key ที่ระบุไว้
GET_NETWORK	✎	wifi: 5 ช่องเสมอ [{"slot":1,"ssid":"...", "dhcp":true[, "ip", "gateway", "subnet", "dns"]}, { "slot":2, "ssid":null}, ...], ota_base ไม่มีรหัสผ่าน ต้อง AUTH (ดูหัวข้อ Wi-Fi 5 ช่อง)
GET_SETTINGS		เนื้อหาของ event SETTINGS แบบของตัวเอง; network (Wi-Fi 5 ช่องแบบเดียวกับ GET_NETWORK) มีหลัง AUTH
MARK_DELIVERED	✎ trip_id	trip_id, trip_date, trip_number, rows: แอปส่ง trip ที่จบแล้วให้ server เอง แล้ว กล่องจะไม่อัปเดตโหลดซ้ำ (ได้ ALREADY_ACTIVE ถ้า trip ยังวิ่ง)
GET_TRIP_SUMMARY	trip_id	trip_id และเกณฑ์, samples, min, max, alarms, stopped
READ_LOG_CHUNK	trip_id, from (seq), max (1-16)	records, next
GET_STORAGE_STATUS		sectors, used, free, trips, days_left
SELF_TEST		health: อุปกรณ์แต่ละตัวและสถานะ
REBOOT	✎	(ตอบก่อน แล้วกล่องรีเซ็ต)
GET_SYNC_STATUS		wifi (เชื่อมต่อ, ssid, rssi, known: ชื่อที่รู้จัก), server (broker, pending แถวค้าง, last_ack_s)
SYNC_NOW	✎	อัปเดตเลยแทนที่จะรอรอบถัดไป
SET_WIFI	✎ slot (1-5, ไม่ใส่ก็ได้), ssid, pass (ว่างสำหรับ เครือข่ายเปิด; ไม่ใส่ได้เมื่อ ssid ในช่องไม่ เปลี่ยน), dhcp, ip, gateway, subnet, dns	เพิ่มเครือข่าย หรือเปลี่ยนรหัสผ่านและ/หรือที่อยู่ (DHCP หรือ IP คงที่ แบบ eTEMP) จำ ได้ 5 เครือข่าย เชื่อมตัวที่สัญญาณแรงสุดในระยะ
DEL_WIFI	✎ slot (1-5) หรือ ssid	ลบเครือข่าย (ช่องนั้นว่าง ช่องอื่นไม่ขยับ)
GET_USB_SNAPSHOT_STATUS		NOT_SUPPORTED จนกว่าจะทำการเรื่อง USB drive

Wi-Fi 5 ช่อง (ตกลง 7 ต.ค.)

กล่องจำ Wi-Fi ได้ 5 ช่อง ตำแหน่งคงที่ ช่องที่ลบแล้วเป็นช่องว่างและช่องอื่นไม่ขยับ แอปจึงอ้าง "ช่อง 3" แล้วได้เครือข่ายเดิมเสมอจนกว่าจะแก้ GET_NETWORK (และ network ใน event SETTINGS) ส่งครบทั้ง 5 ช่อง:

```
{
  "id":10,"ok":true,"wifi":[
    {"slot":1,"ssid":"RDE_2.4GHz","dhcp":true},
    {"slot":2,"ssid":null},
    {"slot":3,"ssid":"Warehouse","dhcp":false,"ip":"192.168.1.50","gateway":"192.168.1.1","subnet":"255.255.255.0","dns":""},
    {"slot":4,"ssid":null},
    {"slot":5,"ssid":null}],
  "ota_base":null
}
```

แอปแก้หรือลบช่องไหนก็ได้ ส่งใน APPLY_CONFIG ครั้งเดียว (ทำ del ก่อน add เสมอ):

ผู้ใช้ทำ	ส่ง
ลบช่อง 3	"wifi":{"del":["3"]}
ใส่เครือข่ายในช่องว่าง	"wifi":{"add":[{"slot":2,"ssid":"Shop","pass":"รหัส","dhcp":true}]} ต้องมี ssid และ pass
เปลี่ยน IP หรือ DHCP ของช่อง	"add":[{"slot":3,"dhcp":true}] ไม่ต้องส่ง ssid หรือ pass กล่องใช้ของเดิมในช่อง
เปลี่ยนรหัสผ่านของช่อง	"add":[{"slot":3,"pass":"รหัสใหม่"]}
เปลี่ยนชื่อเครือข่ายในช่อง	"add":[{"slot":3,"ssid":"ชื่อใหม่","pass":"รหัส"}] ต้องมี pass เสมอ (ชื่อเปลี่ยนแล้วรหัสเก่าใช้ไม่ได้) และช่องกลับเป็น DHCP จนกว่าจะส่ง ip

- **ไม่ส่งรหัสผ่านกลับมา:** แอปจึงดูรหัสเดิมไม่ได้ ถ้าไม่เปลี่ยนรหัส ก็ไม่ต้องส่ง pass
- **ห้ามซ้ำ:** เครือข่ายเดียวกันอยู่ได้ช่องเดียว ได้ error that network is already in another slot
- **ผลลัพธ์รายการ:** คำตอบมี applied และ errors แยกรายช่อง เช่น "wifi slot 3" สำเร็จ ส่วนตัวที่ผิดพลาดบอกเหตุผล
- ส่ง slot ที่เกิน 1-5 ได้ BAD_ARGS (slot 1..5) และลบช่องที่ว่างอยู่ได้ that slot is empty
- กล่องเชื่อมเครือข่ายที่สัญญาณแรงสุดในระยะ ไม่มีลำดับความสำคัญตามช่อง

แอปเห็นค่าที่เปลี่ยนในกล่องอย่างไร

กล่องส่งข้อมูลใหม่ให้เอง แอปไม่ต้องอ่านซ้ำ:

ข้อมูล	กล่องส่งผ่าน	เมื่อไร
ค่าตั้งทั้งหมด และ network (Wi-Fi 5 ช่อง, ota_base)	event SETTINGS ที่ช่อง EVENT	ทันทีที่แอป subscribe, ทันทีที่ session ผ่าน AUTH (เพราะ network มีหลัง AUTH เท่านั้น) และทุกครั้งที่ค่าตั้งเปลี่ยน
อุณหภูมิ แบต สถานะ trip และ alarm	STATUS (notify)	เมื่อ trip เริ่มหรือจบ, alarm เปลี่ยน, ได้รับทราบ alarm และหลังบันทึกแต่ละครั้ง
trip, alarm	event TRIP_START, TRIP_STOP, ALARM_RAISE, ALARM_CLEAR, ALARM_ACK	ทันทีที่เกิด

ค่าตั้งเปลี่ยนจากที่ไหนก็ได้ ทั้งแอป server (เอกสาร config ทาง MQTT) และ console ของกล่อง SETTINGS ส่งให้ทุกกรณี

ข้อควรทำในแอป:

- **ต้อง subscribe ช่อง EVENT และ STATUS** จึงจะได้ข้อความเหล่านี้ ถ้าไม่ได้เชื่อมตอนค่าเปลี่ยน แอปจะได้ค่าล่าสุดทันทีที่เชื่อมแล้ว subscribe
- **event เป็นแบบ notify ไม่รับประกันว่าได้ครบ** ถ้าแอปหลุดชั่วคราวหรือไม่แน่ใจ ให้เรียก GET_SETTINGS ขอค่าล่าสุด
- **SETTINGS ยาว 1-2 KB** มาเป็นหลาย fragment แอปต้องต่อให้ครบ (บิต FIRST/LAST) ก่อนอ่านเป็น JSON
- **สถานะ Wi-Fi สด** (ต่ออยู่ไหม, ชื่อเครือข่าย, สัญญาณ rssi , แถวที่ค้างส่ง) กล่อง**ไม่ส่งเมื่อเปลี่ยน** ให้แอปเรียก GET_SYNC_STATUS เป็นระยะ เช่นทุก 5-10 วินาทีตอนเปิดหน้า Wi-Fi อ่านได้โดยไม่ต้อง AUTH

GET_STATUS :

```
{
  "id": 3,
  "ok": true,
  "time": {
    "utc": 1791357844,
    "quality": "ntp",
    "boot": 42
  },
  "temp": {
    "ok": true,
    "c": 4.25
  },
  "trip": {
    "active": true,
    "trip_id": "95518e06-1b3b-494f-b4c5-284ae94f5424",
    "trip_date": "20261007",
    "trip_number": 2,
    "samples": 12,
    "min": 3.5,
    "max": 5.25,
    "alarms": [
      "TEMP_HIGH"
    ],
    "acked": false,
    "alarms_raised": 1,
    "last_alarm": {
      "type": "HIGH",
      "utc": 1791358100
    },
    "power": {
      "soc": 78,
      "mv": 3987,
      "ma": -12,
      "charge": "none",
      "external": false
    },
    "gnss": {
      "fix": false,
      "lat": null,
      "lon": null,
      "age_s": null
    },
    "storage": {
      "used_pct": 1
    },
    "fw": {
      "ver": "0.7.0-dev.35",
      "slot": "ota_0"
    },
    "net": {
      "connected": true,
      "ssid": "Office",
      "rssi": -59,
      "ip": "192.168.1.147",
      "mac": "28:84:85:27:9A:74"
    }
  }
}
```

trip ใน STATUS เมื่อไม่มี trip ว่าจะมี trip_id ของ trip ล่าสุดที่จบ (active เป็น false) หรือ null ทั้งสามช่องถ้ายังไม่เคยมี fw คือ firmware ที่รันอยู่และ slot OTA ที่รัน; net.ssid, rssi และ ip มีเฉพาะตอนเชื่อมต่อ trip มี alarms_raised (จำนวนทั้ง trip) และ last_alarm ({"type": "HIGH", "utc": ...} หรือ null) ด้วย เพื่อให้ server ที่ติดต่อกล่องไม่ได้รับรู้ว่ามี alarm เกิดและหายไปแล้วระหว่างนั้น

READ_LOG_CHUNK ส่งแฉตามทีกล่องเก็บจริง เป็น JSON ละแฉ {"seq": 5, "type": 16, "data": "<base64>"} โดย data คือแฉตามผังใน src/record.h (record format 3: type 16 ทุก record เป็น ROW คอลัมน์ชุดเดียวกับที่ server ได้ ดูข้อ 6) next คือ seq ที่ขอต่อ หรือ null เมื่อหมด แอปเก็บ **ไบต์ตามผัง** ไม่ใช้การตีความของกล่อง แอปรุ่นเก่าจึงไม่ทำ record จาก firmware ใหม่หาย แต่ยังไม่เข้าใจ

แฉแรก (seq 0, TRIP_START) มี header ของ trip ต่อท้าย 71 ไบต์ (format 5) ซึ่งมี UUID, วัน และเลขรัน ผังอยู่ใน src/record.h

5. คำถามที่ทีมแอปต้องตอบ

- 1. รับหรือขอแก้ UUID, รูปแบบ fragment และ JSON
- 2. แตะเพื่อยืนยันตัว (ข้อ 2): ช่อง key ในแท็ก และ AUTH ด้วย HMAC-SHA256 กับ nonce ของแต่ละการเชื่อมต่อ
- 3. STATUS ควรอ่านได้โดยไม่ต้อง AUTH โหม (มีอุณหภูมิและตำแหน่ง)
- 4. record ใน NFC: เก็บ {"sn", "ble", "key"} โหม และ URL สำหรับแบนเนอร์คืออะไร
- 5. แอปรับ trip_id (UUID), trip_date และ trip_number แทน trip แบบตัวเลขได้ไหม และจะแสดงให้ผู้ใช้ได้อย่างไร

6. Server: MQTT (สำหรับทีม server)

สถานะ: เสนอ ปรับปรุง 7 ต.ค. 2026 ผังกล่องเสร็จและทำงานกับ broker ของทีมแล้ว ผัง server ที่ยังขาดคือ ACK

กล่องต่อ broker ด้วย client id <sn> (SN จากโรงงาน เช่น mCDV1-L0169-1069-002 หรือ mCOLD-xxxx จนกว่าจะตั้ง SN) ตามแบบ eTEMP (ตกลง 5 ต.ค.): **สิ่งที่กล่อง publish อยู่ที่ mco1d/<sn>/** และ **สิ่งที่กล่อง subscribe อยู่ที่ mco1d/v1/<sn>/** ส่วน v1 คือเวอร์ชันของคำสั่งที่กล่องเข้าใจ กล่องที่เข้าใจ v2 จะ subscribe ที่นั่น และทั้งสองรุ่นอยู่บน broker เดียวกันได้ระหว่างทยอยอัปเดต

Topic	ทิศ	QoS	Retain	เนื้อหา
mco1d/<sn>/rec	กล่อง → server	1	ไม่	ชุดแฉของ trip ที่จบแล้ว
mco1d/v1/<sn>/ack	server → กล่อง	1	ไม่	{"trip_id": "<uuid>", "upto": S}
mco1d/<sn>/status	กล่อง → server	0	ใช่	ผลของ GET_STATUS (ข้อ 4)
mco1d/<sn>/online	กล่อง → server	1	ใช่	"1" ขณะเชื่อมต่อ; "0" เมื่อจบรอบตอนใช้แบตเตอรี่หรือจาก broker (last will) ถ้ากล่องหลุด
mco1d/v1/<sn>/firmware	server → กล่อง	1	ใช่	ชื่อไฟล์ที่จะติดตั้ง เช่น mCOLD_0.7.1.bin
mco1d/v1/<sn>/config	server → กล่อง	1	ใช่	เอกสารการตั้งค่า (docs/device-settings.md) ใช้หนึ่งครั้งต่อหนึ่ง rev

Topic	ทิศ	QoS	Retain	เนื้อหา
<code>mcold/<sn>/config/state</code>	กล่อง → server	1	ใช่	ผล: rev, applied, errors, state
<code>mcold/<sn>/ota/state</code>	กล่อง → server	1	ใช่	ผลการอัปเดต (ด้านล่าง)

subscribe `mcold/+/rec` ได้เแของทุกกล่อง

ชุดแถว (batch)

```
{
  "sn": "mCDV1-L0169-1069-002",
  "trip_id": "95518e06-1b3b-494f-b4c5-284ae94f5424",
  "trip_date": "20261007",
  "trip_number": 2,
  "part": 1,
  "parts": 1,
  "from": 0,
  "to": 3,
  "last": true,
  "rows": [
    {
      "trip_id": "95518e06-1b3b-494f-b4c5-284ae94f5424",
      "trip_date": "20261007",
      "trip_number": 2,
      "seq": 0,
      "timestamp": 1791357844,
      "event": "TRIP_START",
      "temp": 25.83,
      "tempmin": 2,
      "tempmax": 8,
      "alarm": "",
      "timeok": true,
      "gnssstate": "none",
      "latitude": null,
      "longitude": null,
      "motion": 0,
      "battery": 99,
      "internet": "online",
      {
        "trip_id": "...",
        "seq": 1,
        "timestamp": 1791357845,
        "event": "SAMPLE",
        ...
      },
      ...
    }
  ]
}
```

แถวใน MQTT ไม่มี sn, utc และ detail (ตกลง 7 ต.ค.): sn อยู่ที่หัวชุดและใน topic อยู่แล้ว, timestamp เป็นวินาที Unix (ว่างเป็น null เมื่อ timeok เป็น false), ส่วน detail ไม่ส่ง ไฟล์ CSV ยังมีครบทุกคอลัมน์ (sn, timestamp ข้อความเวลาท้องถิ่น, utc, detail)

อัปโหลดเฉพาะ trip ที่จบแล้ว (ตกลง 5 ต.ค. docs/trip-data-flow.md): ระหว่างที่ trip วิ่ง แถวอยู่ในกล่อง server ได้แค่ status เมื่อจบแล้ว จึงส่งเป็นชุดละ 20 แถว trip เก่าก่อน ชุดที่ k มี seq 20(k-1) ถึง 20k-1 เสมอ parts คือจำนวนชุดทั้งหมด และ last ระบุชุดที่มี TRIP_STOP trip ที่แอปส่งให้ server แล้ว (MARK_DELIVERED) จะไม่ถูกอัปโหลด

คีย์ของแต่ละแถวคือ (trip_id, seq) และ sn อยู่ในทุกแถวและในชุดเพื่อผูก trip กับกล่อง trip_id เป็น UUID ที่ไม่ซ้ำกันทั่วโลก แต่ server ควรเก็บ sn คู่ไปด้วย (และตรวจว่า sn ใน topic ตรงกับ sn ในชุด)

ขนาด: แถว JSON ละประมาณ 300 ไบต์ ชุดละ 20 แถวจึงราว 6 KB **broker ต้องรับข้อความได้อย่างน้อย 8 KB**

ทุกแถวมีคอลัมน์ชุดเดียวกันไม่ว่าจะเป็น sample หรือ event (record format 3) แต่ละแถวบอกสถานะของกล่อง ณ ขณะนั้น server เก็บตารางเดียวได้ และ CSV ที่คนเปิดก็คอลัมน์เดียวกัน (คำสั่ง trip csv ใน console พิมพ์ออกมา src/logrow.h เป็นตัวกำหนด)

คอลัมน์	ความหมาย
<code>trip_id</code>	UUID ของ trip ที่กล่องสร้าง
<code>trip_date</code>	วันที่ท้องถิ่นตอนเริ่ม trip YYYYMMDD
<code>trip_number</code>	เลขรันของ trip ในวันนี้ เริ่มที่ 1
<code>seq</code>	ลำดับแถวใน trip เริ่มที่ 0 (trip_id, seq คือคีย์)
<code>sn</code>	serial number ของกล่อง (มีใน CSV; ใน MQTT อยู่ที่หัวชุด)
<code>timestamp</code>	MQTT: วินาที Unix (UTC) เป็นตัวเลข · CSV: เวลาท้องถิ่น hh:mm:ss DD/MM/YYYY (ค่า tz_offset_min ปกติ +07:00)
<code>utc</code>	เฉพาะ CSV: ช่วงเวลาเดียวกันเป็นวินาที Unix
<code>event</code>	แถวนี้คืออะไร (ตารางด้านล่าง)
<code>temp</code>	°C หลังปรับเทียบ; null เมื่อ probe ไม่ให้ค่า
<code>tempmin / tempmax</code>	เกณฑ์ alarm ของ trip ตั้งแต่เริ่ม
<code>alarm</code>	alarm ที่ค้างหลังแถวนี้ HIGH, LOW, PROBE, BATTERY คั่นด้วย ว่าง = ไม่มี
<code>timeok</code>	false เมื่อกล่องไม่รู้เวลา ตอนนั้น timestamp / utc เป็น null และใช้ boot + up_s เรียงลำดับแทน
<code>gnssstate</code>	fix (ภายในรอบบันทึกล่าสุด), last (ตำแหน่งเก่ากว่านั้น), none (ยังไม่เคยได้ตำแหน่งตั้งแต่บูต)
<code>latitude / longitude</code>	องศา; null เมื่อ none

คอลัมน์	ความหมาย	
motion	จำนวนครั้งที่ขยับตั้งแต่แถวก่อน	
battery	แบตเตอรี่ %; null เมื่อไม่ทราบ	
internet	ผลการส่งรอบล่าสุด: online , wifi_only (มี Wi-Fi แต่ไม่ถึง broker), offline	
detail	เฉพาะ CSV: รายละเอียดของ event เป็นคำ (ด้านล่าง) ส่วนใหญ่ว่าง	
event	เกิดเมื่อ	detail
TRIP_START / TRIP_STOP	trip เริ่ม / จบ	
SAMPLE	ทุกรอบบันทึก	
ALARM_HIGH / ALARM_LOW	เกิน tempmax / ต่ำกว่า tempmin นานกว่า dwell	
ALARM_PROBE	ไม่มีอุณหภูมิเกิน 1 นาที	
BATTERY_LOW	แบตเตอรี่ต่ำกว่า 15%	%
ALARM_CLEAR	alarm หาย	ชนิดที่หาย
ALARM_ACK	มีคนรับทราบ alarm	alarm ที่รับทราบ
PROBE_FAULT / PROBE_OK	probe หยุดตอบ / กลับมาตอบ	สาเหตุ
USB_IN / USB_OUT	เสียบ / ถอดไฟนออก	
POWER_ON	trip ทำต่อหลังรีเซ็ต	สาเหตุของการรีเซ็ต
POWER_OFF	แบตเตอรี่หมด กล้องตัดตัวเอง	mV
TIME_SET	มีการตั้งเวลา	เวลาก่อนตั้ง
DATA_LOST	trip เก่าถูกลบเพราะพื้นที่เต็ม	เลขของ trip นั้น (ของกล้อง)
SHOCK	สำรอง: กระแทกเกินเกณฑ์ (ยังไม่ได้ตั้งเกณฑ์)	

การขยับธรรมดาไม่เป็นแถว นับรวมในคอลัมน์ `motion` trip ที่บันทึกก่อน header format 5 (ไม่มี `trip_id`) ไม่อัปเดต

ACK: สิ่งที่ server ต้องทำ

กล้องไม่ลอบอะไรจนกว่า server จะบอกว่าเก็บแล้ว **PUBACK ของ MQTT ไม่นับ** เพราะแปลว่า broker รับ ไม่ใช่ server เก็บ

- 1. เก็บแถวของชุดนั้นลงฐานข้อมูลให้เรียบร้อย **คีย์คี่ (trip_id, seq)** แถวที่มาซ้ำ (กล้องส่งซ้ำเมื่อไม่ได้ ACK) ต้องเก็บครั้งเดียว
- 2. จากนั้น publish ที่ `mco1d/v1/<sn>/ack` :

```
{ "trip_id": "95518e06-1b3b-494f-b4c5-284ae94f5424", "upto": 15 }
```

แปลว่า**ทุกแถวของ trip นี้ตั้งแต่ seq 0 ถึง 15 เก็บแล้ว** ต้องต่อเนื่อง ตอบ upto เฉพาะเมื่อแถวที่น้อยกว่านั้นเก็บครบแล้วเท่านั้น

กล้องตรวจ ACK ทุกครั้งกับ log ของตัวเอง ถ้า `trip_id` ไม่มีในกล้อง หรือ upto เกินแถวสุดท้าย จะไม่รับและไม่ทำเครื่องหมายอะไร (ACK แบบเก่าที่ใช้ "trip": ตัวเลขจะถูกปฏิเสธ) ค่านี้เดินหน้าอย่างเดียวก ACK ซ้ำหรือมาซ้ำจึงไม่เสียหาย

ตอนใช้ไฟนอก และตอนใช้แบตเตอรี่

ตอนเสียบไฟ กล้องต่อ broker ค้างไว้ ส่งชุดแล้วรอ ACK 15 วินาที แล้วส่งใหม่ โดยเพิ่มเวลารอเป็นสองเท่า สูงสุด 5 นาทีถ้า server เจียบ และกลับเป็น 15 วินาทีเมื่อได้ ACK ส่ง `status` เมื่อมีการเปลี่ยน ทุก 5 นาที และทันทีเมื่อ broker ตอบอีกครั้ง

ตอนใช้แบตเตอรี่ กล้องหลับระหว่างบันทึกและปิด Wi-Fi ระหว่างที่ trip วิ่ง หรือมีแถวของ trip ที่จบแล้วรอส่ง จะเชื่อมต่อเป็น **รอบสั้นๆ** หนึ่งครั้งต่อ `upload_period_s` (ค่าเริ่มต้น 300 วินาที): เปิด Wi-Fi → เชื่อม broker → ส่ง `status` → ส่งชุดแถว → ส่ง "0" ที่ `online` แล้วตัดการเชื่อมต่อ ในแต่ละรอบรอ ACK ชุดละไม่เกิน **10 วินาที** ACK ที่มาหลังจบรอบจะหายไป (กล้องใช้ clean session) กล้องส่งชุดเดิมอีกในรอบหน้า และ server ที่ใช้คีย์ (`trip_id`, `seq`) แค ACK ซ้ำ ถ้ารอบหนึ่งไม่ได้ ACK เลย กล้องเว้นรอบถัดไปเป็น 10, 20, 40 ... นาที สูงสุด 4 ชั่วโมง เพราะ server ที่ไม่ตอบจะทำให้กล้องเสียพลังงานวิทยุทุก 5 นาที

สำหรับ server:

- **ACK ให้เร็ว** ทุกวินาทีที่ server ใช้คือวิทยุที่เปิดค้างในทุกกล่อง ACK ภายใน 1–2 วินาทีทำให้รอบหนึ่งใช้เวลาราว 5 วินาที
- ดูว่ากล้องยังติดต่อได้จาก **เวลาของ status ล่าสุด** ไม่ใช่จาก `online` เพราะกล้องที่ใช้แบตเตอรี่เป็น "0" เกือบตลอดและถือว่าปกติ

เมื่อ log เต็ม กล้องลบ trip ที่ server ACK ครบแล้วก่อน ไม่เสียข้อมูล และถ้าไม่มีเลยจึงลบ trip เก่าสุดที่ยังไม่ครบ พร้อมบันทึกการสูญหาย (DATA_LOST)

อัปเดต firmware (OTA)

ทดสอบเมื่อ 4 ต.ค. 2026 แบบเดียวกับ eTEMP V2: server ระบุชื่อไฟล์ แล้วกล้องดาวน์โหลดเอง

- วางไฟล์ไว้ใน file server ตามโฟลเดอร์ที่กล้องรู้ (URL ฐานเก็บใน NVS ตั้งด้วย ota base URL ใน console) ผลการ build อยู่ที่ .pio/build/mcold/mcCOLD_<version>.bin
- publish ชื่อไฟล์ที่ mcold/v1/<sn>/firmware แบบ **retain** (กล้องที่ใช้แบบหลักเก็บตลอดและจะเห็นตอนรอบถัดไป) ใส่ URL เต็ม https://... ก็ได้
- ดูผลที่ mcold/<sn>/ota/state :

```
{ "ver": "0.7.1", "state": "downloading", "pct": 40 }
{ "ver": "0.7.1", "state": "rebooting", "from": "0.7.0" }
{ "ver": "0.7.1", "state": "ok", "from": "0.7.0" }
{ "file": "mCOLD_0.7.1.bin", "state": "failed", "reason": "..." }
{ "file": "mCOLD_0.7.1.bin", "state": "deferred", "reason": "trip" }
{ "ver": "0.7.1", "state": "rolled_back", "running": "0.7.0" }
{ "file": "...", "ver": "0.7.0", "state": "skipped", "reason": "not newer", "running": "0.7.0" }
```

1. ได้ ok แล้วลบข้อความ retain (publish payload ว่างแบบ retain) ชื่อที่กล้องเคยทำแล้วจะถูกข้ามอยู่แล้ว การปล่อยไว้เสียแค่ไม่กี่ไบต์ต่อรอบ กล้องตรวจเองทั้งหมด server ไม่ต้องรู้อะไร: ส่วนหัวของไฟล์ (โครงการเดียวกัน เวอร์ชันใหม่กว่า), SHA-256 และลายเซ็น (RSA-3072 ตรงกับ key ของไฟล์ที่รันอยู่ ไฟล์ที่ไม่ได้เซ็นด้วย key ของโครงการถูกปฏิเสธ) หลังรับชุดกล้องต้องต่อ broker ได้ภายใน 3 นาที ไม่เช่นนั้น bootloader ย้อนไปไฟล์เดิมและรายงาน rolled_back กล้องจะรอ (deferred) ขณะมี trip วิ่งหรือแบตเตอรี่ต่ำกว่า 30% แล้วทำเองเมื่อพร้อม ตอนใช้แบบที่ไม่ทำอะไรต้องส่ง กล้องยังเช็กอินทุก 6 ชั่วโมงเพื่อให้เห็นข้อความ firmware

คำถามที่ต้องการคำตอบจากทีม server

- รับ topic, รูปแบบแถว (ใช้ trip_id UUID) และ ACK ตามนี้ หรือขอแก้ไข
- เปิด TLS ที่พอร์ต 8883 และ login แยกต่อกล้อง: ที่พอร์ต 1883 ทุกอย่างรวมทั้ง login วิ่งแบบไม่เข้ารหัส และ login ร่วมหนึ่งชุดทำให้ใครถือก็ publish แทนกล้องใดก็ได้ รวมถึง ACK ปลอมที่ทำให้กล้องลบข้อมูลที่ยังไม่ถึง server
- broker จำกัดขนาดข้อความไว้เท่าไร (ต้องไม่ต่ำกว่า 8 KB)